

树与二叉树

数据结构考试复习

2026 年 5 月 28 日

目录

1 基础概念	4
1.1 树的定义	4
1.2 基本术语	4
1.3 树的遍历	4
2 二叉树	4
2.1 定义与性质	4
2.2 满二叉树 vs 完全二叉树	5
2.3 二叉树的存储方式	5
3 Python 实现：二叉树的遍历	5
3.1 结点定义	5
3.2 DFS（深度优先搜索，Depth-First Search）遍历（递归）	6
3.3 DFS（深度优先搜索）非递归（用栈）	6
3.4 BFS（广度优先搜索，Breadth-First Search）层序遍历	7
4 二叉树的重建（已知遍历序列求二叉树）	8
4.1 原理	8
4.2 代码实现	8
4.3 参数范围对照表	11
5 完全二叉树的遍历序列转层序	11
6 二叉搜索树（BST, Binary Search Tree）	13
6.1 性质	13
6.2 代码实现	14
7 平衡二叉树（AVL, Adelson-Velsky and Landis Tree）	15
7.1 性质	15
7.2 四种旋转	16
7.3 代码实现	16
8 哈夫曼树（Huffman Tree, 最优二叉树）	19
8.1 定义	19
8.2 构造规则	19
8.3 性质	19

8.4 代码实现	19
9 堆 (Heap, 优先队列)	21
10 并查集 (Union-Find, 联合查找)	22
11 常用技巧与竞赛模板	22
11.1 求树的高度	22
11.2 求树的直径 (任意两节点最长路径)	23
11.3 判断是否为对称二叉树	23
11.4 判断是否为平衡二叉树	23
11.5 二叉树的镜像翻转	24
11.6 找二叉树的最近公共祖先 (LCA, Lowest Common Ancestor)	24
11.7 二叉树的序列化与反序列化	24
12 笔试常见考点速查	25
13 Python 引用模块汇总 (含常用代码模板)	26
13.1 collections — 队列与字典	26
13.2 heapq — 堆	26
13.3 sys — 快速输入	27
13.4 itertools — 迭代工具	27
13.5 functools — 函数式工具	28
13.6 math — 数学运算	28
14 考试快速参考模板	28

1 基础概念

1.1 树的定义

树是 n ($n \geq 0$) 个结点的有限集合。当 $n = 0$ 时称为**空树**。在任意非空树中：

- 有且仅有一个特定的称为**根 (root)** 的结点
- 其余结点可分为 m ($m > 0$) 个互不相交的有限集合，每个子集又是一棵树，称为根的**子树 (subtree)**

1.2 基本术语

结点的度 (degree) 结点拥有的子树个数

树的度 树中各结点度的最大值

叶子 (leaf) 度为 0 的结点

分支结点 (branch) 度 > 0 的结点

孩子 (child) / 双亲 (parent) 结点的子树的根称为该结点的孩子，该结点称为孩子的双亲

兄弟 (sibling) 同一双亲的孩子之间互称兄弟

祖先 (ancestor) 从根到该结点路径上的所有结点

子孙 (descendant) 结点的子树中的任意结点

层次 (level) 根为第 1 层，其孩子为第 2 层，以此类推

深度 (depth) 树中结点的最大层次

森林 (forest) m ($m \geq 0$) 棵互不相交的树的集合

1.3 树的遍历

- **先根遍历**：先访问根，再依次先根遍历每棵子树
- **后根遍历**：先依次后根遍历每棵子树，再访问根
- **层序遍历**：从上到下、从左到右依次访问每个结点

2 二叉树

2.1 定义与性质

二叉树是每个结点至多有两棵子树的树结构，子树有左右之分。

- 性质 1: 在第 i 层最多有 2^{i-1} 个结点 ($i \geq 1$)。
- 性质 2: 深度为 k 的二叉树最多有 $2^k - 1$ 个结点 ($k \geq 1$)。
- 性质 3: 叶子数 n_0 与度为 2 的结点数 n_2 满足 $n_0 = n_2 + 1$ 。
- 性质 4: n 个结点的完全二叉树深度为 $\lfloor \log_2 n \rfloor + 1$ 。

2.2 满二叉树 vs 完全二叉树

特征	满二叉树	完全二叉树
定义	深度 k 且含 $2^k - 1$ 个结点	除最后一层外全满，最后一层靠左排列
结点数	$2^k - 1$	不确定
叶子位置	都在最后一层	可能在最后两层
编号关系	完全二叉树的特例	可用 1-indexed 数组存储

2.3 二叉树的存储方式

1. 顺序存储（数组，适用于完全二叉树）

完全二叉树按层序编号 $1, 2, \dots, n$ ，编号 i 的结点：

- 左孩子: $2i$ (若 $2i \leq n$)
- 右孩子: $2i + 1$ (若 $2i + 1 \leq n$)
- 双亲: $\lfloor i/2 \rfloor$ (若 $i \geq 2$)

2. 链式存储（指针）

每个结点包含：值 + 左孩子指针 + 右孩子指针。

3 Python 实现：二叉树的遍历

3.1 结点定义

```

1 from collections import deque
2
3
4 class TreeNode:
5     """二叉树节点类"""
6     def __init__(self, x):
7         self.val = x    # 节点值
8         self.left = None # 左子节点
9         self.right = None # 右子节点

```

3.2 DFS（深度优先搜索，Depth-First Search）遍历（递归）

```
1 def preorder(root):
2     """前序遍历：根 -> 左 -> 右"""
3     if not root:
4         return []
5     return [root.val] + preorder(root.left) + preorder(root.right)
6
7
8 def inorder(root):
9     """中序遍历：左 -> 根 -> 右"""
10    if not root:
11        return []
12    return inorder(root.left) + [root.val] + inorder(root.right)
13
14
15 def postorder(root):
16     """后序遍历：左 -> 右 -> 根"""
17     if not root:
18         return []
19     return postorder(root.left) + postorder(root.right) + [root.val]
```

3.3 DFS（深度优先搜索）非递归（用栈）

```
1 def preorder_iter(root):
2     """前序遍历 非递归：根 -> 左 -> 右"""
3     if not root:
4         return []
5     res, stack = [], [root]
6     while stack:
7         node = stack.pop()    # 弹出栈顶
8         res.append(node.val) # 访问根
9         if node.right:
10            stack.append(node.right) # 右先入栈（后出）
11        if node.left:
12            stack.append(node.left) # 左后入栈（先出）
13    return res
14
15
16 def inorder_iter(root):
17     """中序遍历 非递归：左 -> 根 -> 右"""
18     res, stack = [], []
19     cur = root
20     while cur or stack:
21         # 一直往左走，沿途入栈
22         while cur:
```

```

23         stack.append(cur)
24         cur = cur.left
25         # 弹出栈顶，访问，再转向右子树
26         cur = stack.pop()
27         res.append(cur.val)
28         cur = cur.right
29     return res
30
31
32 def postorder_iter(root):
33     """后序遍历 非递归（双栈法）：左 -> 右 -> 根"""
34     if not root:
35         return []
36     res, stack = [], [root]
37     while stack:
38         node = stack.pop()
39         res.append(node.val)
40         if node.left:
41             stack.append(node.left)
42         if node.right:
43             stack.append(node.right)
44     return res[::-1] # 前序的逆序 = 后序

```

3.4 BFS（广度优先搜索，Breadth-First Search）层序遍历

```

1 def levelorder(root):
2     """层序遍历：从上到下，从左到右（使用队列）"""
3     if not root:
4         return []
5     q = deque([root]) # 初始化队列，根节点入队
6     res = []          # 存储遍历结果
7     while q:
8         node = q.popleft() # 队首出队
9         res.append(node.val) # 访问当前节点
10        if node.left:
11            q.append(node.left) # 左子节点入队
12        if node.right:
13            q.append(node.right) # 右子节点入队
14    return res
15
16
17 def levelorder_by_level(root):
18     """层序遍历（分层输出，每层一行）"""
19     if not root:
20         return []
21     q = deque([root])

```

```

22     res = []
23     while q:
24         level_vals = []          # 当前层的节点值
25         for _ in range(len(q)): # 一次性处理当前层所有节点
26             node = q.popleft()
27             level_vals.append(node.val)
28             if node.left:
29                 q.append(node.left)
30             if node.right:
31                 q.append(node.right)
32         res.append(level_vals) # 将当前层加入结果
33     return res

```

4 二叉树的重建（已知遍历序列求二叉树）

4.1 原理

- 前序 + 中序：前序第一个为根 → 在中序中定位根 → 划分左右子树
- 中序 + 后序：后序最后一个为根 → 在中序中定位根 → 划分左右子树
- 前序 + 后序：结果不唯一（单子树时无法区分左右）

4.2 代码实现

```

1  from collections import deque
2
3
4  class TreeNode:
5      """二叉树节点类"""
6      def __init__(self, x):
7          self.val = x
8          self.left = None
9          self.right = None
10
11
12  # ===== 前序 + 中序 =====
13  def build_from_pre_in(preorder, inorder):
14      """
15      根据前序遍历和中序遍历构建二叉树
16      核心思想：
17          前序第一个节点是根
18          中序中根左边是左子树，右边是右子树
19      """
20      if not preorder or not inorder:

```



```

21     return None
22
23     n = len(preorder)
24     # 哈希表优化：用字典记录每个值在中序中的位置，O(1)查找
25     in_map = {}
26     for i in range(len(inorder)):
27         in_map[inorder[i]] = i
28
29     def dfs(pre_l, pre_r, in_l, in_r):
30         """递归建树，参数范围是闭区间 [start, end]"""
31         if pre_l > pre_r:
32             return None
33
34         # 前序第一个节点是根
35         root_val = preorder[pre_l]
36         root = TreeNode(root_val)
37
38         # 在中序中找到根的位置
39         in_root = in_map[root_val]
40         # 左子树的节点个数
41         left_size = in_root - in_l
42
43         # 递归构建左右子树
44         root.left = dfs(pre_l + 1, pre_l + left_size, in_l, in_root - 1)
45         root.right = dfs(pre_l + left_size + 1, pre_r, in_root + 1, in_r)
46         return root
47
48     return dfs(0, n - 1, 0, n - 1)
49
50
51 # ===== 简易版（不用哈希表） =====
52 def build_from_pre_in_simple(preorder, inorder):
53     """
54     简易版：不用哈希表，直接用 index() 查找
55     简单易记，但时间复杂度 O(N^2)
56     """
57     if not preorder or not inorder:
58         return None
59     root_val = preorder[0] # 前序第一个为根
60     root = TreeNode(root_val)
61     idx = inorder.index(root_val) # 在中序中找根的位置
62     root.left = build_from_pre_in_simple(preorder[1:1 + idx], inorder[:idx])
63     root.right = build_from_pre_in_simple(preorder[1 + idx:], inorder[idx + 1:])
64     return root
65

```

```

66
67 # ===== 中序 + 后序 =====
68 def build_from_in_post(inorder, postorder):
69     """
70     根据中序遍历和后序遍历构建二叉树
71     核心思想：
72         后序最后一个节点是根
73         中序中根左边是左子树，右边是右子树
74     """
75     if not inorder or not postorder:
76         return None
77
78     n = len(inorder)
79     # 哈希表优化
80     in_map = {}
81     for i in range(len(inorder)):
82         in_map[inorder[i]] = i
83
84     def dfs(in_l, in_r, post_l, post_r):
85         if post_l > post_r:
86             return None
87
88         root_val = postorder[post_r] # 后序最后一个为根
89         root = TreeNode(root_val)
90         in_root = in_map[root_val]
91         left_size = in_root - in_l
92
93         root.left = dfs(in_l, in_root - 1, post_l, post_l + left_size - 1)
94         root.right = dfs(in_root + 1, in_r, post_l + left_size, post_r - 1)
95         return root
96
97     return dfs(0, n - 1, 0, n - 1)
98
99
100 # ===== 前序 + 后序（不唯一） =====
101 def build_from_pre_post(preorder, postorder):
102     """
103     根据前序遍历和后序遍历构建二叉树
104     注意：结果不唯一！当节点只有左子树或只有右子树时无法区分
105     """
106     if not preorder or not postorder:
107         return None
108
109     n = len(preorder)
110     # 哈希表记录每个值在后序中的位置
111     post_map = {}

```

```

112     for i in range(len(postorder)):
113         post_map[postorder[i]] = i
114
115     def dfs(pre_l, pre_r, post_l, post_r):
116         if pre_l > pre_r:
117             return None
118
119         root = TreeNode(preorder[pre_l])
120         if pre_l == pre_r: # 只有一个节点
121             return root
122
123         # 左子树的根是前序的第二个元素
124         left_root_val = preorder[pre_l + 1]
125         left_root_pos = post_map[left_root_val] # 在后序中的位置
126         left_size = left_root_pos - post_l + 1 # 左子树的节点数
127
128         root.left = dfs(pre_l + 1, pre_l + left_size, post_l, left_root_pos)
129         root.right = dfs(pre_l + left_size + 1, pre_r, left_root_pos + 1, post_r
130                         - 1)
131         return root
132
133     return dfs(0, n - 1, 0, n - 1)

```

4.3 参数范围对照表

构建方式	根节点位置	左子树范围	右子树范围
前序 + 中序	pre[preL]	pre: [preL+1, preL+leftSize] in: [inL, inRoot-1]	pre: [preL+leftSize+1, preR] in: [inRoot+1, inR]
中序 + 后序	post[postR]	in: [inL, inRoot-1] post: [postL, postL+leftSize-1]	in: [inRoot+1, inR] post: [postL+leftSize, postR-1]
前序 + 后序	pre[preL]	pre: [preL+1, preL+leftSize] post: [postL, leftRootPos]	pre: [preL+leftSize+1, preR] post: [leftRootPos+1, postR-1]

5 完全二叉树的遍历序列转层序

原理：完全二叉树可以用 1-indexed 数组（堆式存储）唯一表示：

- 根节点下标为 1
- 节点 i 的左孩子为 $2i$ ，右孩子为 $2i + 1$
- 数组下标顺序 = 层序遍历顺序

对数组下标做对应顺序的遍历，同时按序从给定序列中取值填入数组即可还原层序。

示例（后序 → 层序）：

输入后序: [91, 71, 2, 34, 10, 15, 55, 18]

层序数组下标（完全二叉树）：

层序下标顺序 → 1, 2, 3, 4, 5, 6, 7, 8

后序访问下标顺序: 8 → 4 → 5 → 2 → 6 → 7 → 3 → 1

按后序访问顺序依次填入输入序列：

$\underbrace{91}_8 \rightarrow \underbrace{71}_4 \rightarrow \underbrace{2}_5 \rightarrow \underbrace{34}_2 \rightarrow \underbrace{10}_6 \rightarrow \underbrace{15}_7 \rightarrow \underbrace{55}_3 \rightarrow \underbrace{18}_1$

结果数组（层序）: [18, 34, 55, 71, 2, 10, 15, 91]

三种遍历的差异仅在递归函数中赋值与递归调用的顺序：

- 后序版：左 → 右 → 根（先递归左右子树，最后赋值）
- 前序版：根 → 左 → 右（先赋值，再递归左右子树）
- 中序版：左 → 根 → 右（先递归左子树，赋值，再递归右子树）

```

1  # ===== 后序 -> 层序 =====
2  def postorder_to_level(seq: List[int]) -> List[int]:
3      n = len(seq)
4      tree = [0] * (n + 1)
5      idx = 0
6
7      def dfs(i: int):
8          nonlocal idx
9          if i > n:
10             return
11             dfs(2 * i)          # 左
12             dfs(2 * i + 1)      # 右
13             tree[i] = seq[idx] # 根
14             idx += 1
15
16     dfs(1)

```

```

17     return tree[1:]
18
19
20 # ===== 前序 -> 层序 =====
21 def preorder_to_level(seq: List[int]) -> List[int]:
22     n = len(seq)
23     tree = [0] * (n + 1)
24     idx = 0
25
26     def dfs(i: int):
27         nonlocal idx
28         if i > n:
29             return
30         tree[i] = seq[idx] # 根
31         idx += 1
32         dfs(2 * i)        # 左
33         dfs(2 * i + 1)    # 右
34
35     dfs(1)
36     return tree[1:]
37
38
39 # ===== 中序 -> 层序 =====
40 def inorder_to_level(seq: List[int]) -> List[int]:
41     n = len(seq)
42     tree = [0] * (n + 1)
43     idx = 0
44
45     def dfs(i: int):
46         nonlocal idx
47         if i > n:
48             return
49         dfs(2 * i)        # 左
50         tree[i] = seq[idx] # 根
51         idx += 1
52         dfs(2 * i + 1)    # 右
53
54     dfs(1)
55     return tree[1:]

```

6 二叉搜索树 (BST, Binary Search Tree)

6.1 性质

- 左子树所有结点的值 < 根结点的值

- 右子树所有结点的值 > 根结点的值
- 左右子树也分别是 BST（二叉搜索树）
- 中序遍历 = 递增序列

6.2 代码实现

```

1 class BST:
2     def __init__(self):
3         self.root = None
4
5     def search(self, val):
6         """查找值为 val 的节点"""
7         cur = self.root
8         while cur and cur.val != val:
9             if val < cur.val:
10                 cur = cur.left
11             else:
12                 cur = cur.right
13         return cur
14
15     def insert(self, val):
16         """插入值为 val 的节点"""
17         if not self.root:
18             self.root = TreeNode(val)
19             return
20         cur = self.root
21         while True:
22             if val < cur.val:
23                 if cur.left:
24                     cur = cur.left
25                 else:
26                     cur.left = TreeNode(val)
27                     return
28             elif val > cur.val:
29                 if cur.right:
30                     cur = cur.right
31                 else:
32                     cur.right = TreeNode(val)
33                     return
34             else:
35                 return # 已存在, 不重复插入
36
37     def delete(self, val):
38         """删除值为 val 的节点"""
39         self.root = self._delete(self.root, val)

```

```

40
41 def _delete(self, root, val):
42     if not root:
43         return None
44     if val < root.val:
45         root.left = self._delete(root.left, val)
46     elif val > root.val:
47         root.right = self._delete(root.right, val)
48     else:
49         # 找到要删除的节点
50         if not root.left: # 没有左子树（或空）
51             return root.right
52         if not root.right: # 没有右子树
53             return root.left
54         # 左右子树都有：找右子树的最小值替换
55         min_node = self._min(root.right)
56         root.val = min_node.val
57         root.right = self._delete(root.right, min_node.val)
58     return root
59
60 def _min(self, root):
61     """找以 root 为根的子树的最小节点"""
62     while root.left:
63         root = root.left
64     return root

```

7 平衡二叉树（AVL, Adelson-Velsky and Landis Tree）

7.1 性质

- 任意结点的平衡因子 $BF = h_{\text{left}} - h_{\text{right}} \in \{-1, 0, 1\}$
- 高度 $h \leq 1.44 \log_2(n + 2) - 0.328$
- 查找、插入、删除均为 $O(\log n)$

7.2 四种旋转

失衡类型	情形	旋转方式
LL（左左）	插入在左子树的左子树	右旋
RR（右右）	插入在右子树的右子树	左旋
LR（左右）	插入在左子树的右子树	先左旋后右旋
RL（右左）	插入在右子树的左子树	先右旋后左旋

7.3 代码实现

```
1 class AVLNode:
2     def __init__(self, val):
3         self.val = val
4         self.left = None
5         self.right = None
6         self.height = 1 # 新节点高度为 1（叶子高度）
7
8
9 class AVLTree:
10     def __init__(self):
11         self.root = None
12
13     def _height(self, node):
14         return node.height if node else 0
15
16     def _balance(self, node):
17         """计算平衡因子：左子树高度 - 右子树高度"""
18         return self._height(node.left) - self._height(node.right) if node else 0
19
20     def _rotate_right(self, y):
21         """
22         右旋（LL型）
23
24             y          x
25         /  \        /  \
26        x   T3  -> T1  y
27       /  \        /  \
28      T1 T2        T2 T3
29
30         """
31         x = y.left
32         y.left = x.right
33         x.right = y
34         # 更新高度（先更新 y 再更新 x）
35         y.height = max(self._height(y.left), self._height(y.right)) + 1
36         x.height = max(self._height(x.left), self._height(x.right)) + 1
37         return x
```



```

36
37 def _rotate_left(self, x):
38     """
39     左旋 (RR型)
40
41     x          y
42     / \        / \
43     T1 y  ->  x T3
44     / \        / \
45     T2 T3      T1 T2
46     """
47     y = x.right
48     x.right = y.left
49     y.left = x
50     x.height = max(self._height(x.left), self._height(x.right)) + 1
51     y.height = max(self._height(y.left), self._height(y.right)) + 1
52     return y
53
54 def insert(self, val):
55     """插入值为 val 的节点，保持平衡"""
56     self.root = self._insert(self.root, val)
57
58 def _insert(self, node, val):
59     # 1. 普通 BST 插入
60     if not node:
61         return AVLNode(val)
62     if val < node.val:
63         node.left = self._insert(node.left, val)
64     elif val > node.val:
65         node.right = self._insert(node.right, val)
66     else:
67         return node # 重复值不插入
68
69     # 2. 更新高度
70     node.height = max(self._height(node.left), self._height(node.right)) + 1
71
72     # 3. 检查平衡因子，失衡则旋转
73     bf = self._balance(node)
74
75     # LL: 左子树的左子树插入 -> 右旋
76     if bf > 1 and val < node.left.val:
77         return self._rotate_right(node)
78     # RR: 右子树的右子树插入 -> 左旋
79     if bf < -1 and val > node.right.val:
80         return self._rotate_left(node)
81     # LR: 左子树的右子树插入 -> 先左旋后右旋
82     if bf > 1 and val > node.left.val:

```

```

82         node.left = self._rotate_left(node.left)
83         return self._rotate_right(node)
84     # RL: 右子树的左子树插入 -> 先右旋后左旋
85     if bf < -1 and val < node.right.val:
86         node.right = self._rotate_right(node.right)
87         return self._rotate_left(node)
88
89     return node
90
91     def delete(self, val):
92         """删除值为 val 的节点，保持平衡"""
93         self.root = self._delete(self.root, val)
94
95     def _delete(self, node, val):
96         # 1. 普通 BST 删除
97         if not node:
98             return None
99         if val < node.val:
100             node.left = self._delete(node.left, val)
101         elif val > node.val:
102             node.right = self._delete(node.right, val)
103         else:
104             if not node.left or not node.right:
105                 node = node.left or node.right
106             else:
107                 # 找右子树最小值替换
108                 succ = node.right
109                 while succ.left:
110                     succ = succ.left
111                 node.val = succ.val
112                 node.right = self._delete(node.right, succ.val)
113
114         if not node:
115             return node
116
117         # 2. 更新高度
118         node.height = max(self._height(node.left), self._height(node.right)) + 1
119
120         # 3. 检查平衡
121         bf = self._balance(node)
122
123         if bf > 1 and self._balance(node.left) >= 0:
124             return self._rotate_right(node)
125         if bf < -1 and self._balance(node.right) <= 0:
126             return self._rotate_left(node)
127         if bf > 1 and self._balance(node.left) < 0:

```

```

128         node.left = self._rotate_left(node.left)
129         return self._rotate_right(node)
130     if bf < -1 and self._balance(node.right) > 0:
131         node.right = self._rotate_right(node.right)
132         return self._rotate_left(node)
133
134     return node

```

8 哈夫曼树（Huffman Tree, 最优二叉树）

8.1 定义

带权路径长度（WPL, Weighted Path Length）最小的二叉树：
$$WPL = \sum_{i=1}^n w_i \times l_i$$
，其中 w_i 为权值， l_i 为路径长度。

8.2 构造规则

1. 将所有权值结点作为单结点森林
2. 每次选权值最小的两棵树合并，新根权值为两者之和
3. 重复直到只剩一棵树

8.3 性质

- 只有度为 0 和 2 的结点，没有度为 1 的结点
- $n_0 = n_2 + 1$
- 权值越大的叶子离根越近
- 结构不唯一，但 WPL（带权路径长度）唯一最小

8.4 代码实现

```

1 import heapq
2
3
4 class HuffmanNode:
5     def __init__(self, val, char=''):
6         self.val = val      # 权值
7         self.char = char    # 字符（可选，用于编码）
8         self.left = None

```

```

9         self.right = None
10
11     # 让节点可以比较大小（堆中用到）
12     def __lt__(self, other):
13         return self.val < other.val
14
15
16 def build_huffman_tree(weights):
17     """
18     构造哈夫曼树
19     weights: 权值列表
20     返回根节点
21     """
22     # 将所有权值变成节点，放入最小堆
23     heap = [HuffmanNode(w) for w in weights]
24     heapq.heapify(heap)
25
26     # 每次取最小的两个合并，直到只剩一棵树
27     while len(heap) > 1:
28         a = heapq.heappop(heap) # 最小的
29         b = heapq.heappop(heap) # 第二小的
30         parent = HuffmanNode(a.val + b.val)
31         parent.left = a
32         parent.right = b
33         heapq.heappush(heap, parent)
34
35     return heap[0]
36
37
38 def calc_wpl(root, depth=0):
39     """计算 WPL（带权路径长度）"""
40     if not root.left and not root.right: # 叶子节点
41         return root.val * depth
42     return calc_wpl(root.left, depth + 1) + calc_wpl(root.right, depth + 1)
43
44
45 def huffman_codes(root, prefix=''):
46     """生成哈夫曼编码（向左走为 0，向右走为 1）"""
47     codes = {}
48     if not root.left and not root.right: # 叶子
49         codes[root.char] = prefix or '0'
50         return codes
51     if root.left:
52         codes.update(huffman_codes(root.left, prefix + '0'))
53     if root.right:
54         codes.update(huffman_codes(root.right, prefix + '1'))

```

9 堆（Heap，优先队列）

堆是完全二叉树，可用数组存储。

最小堆：每个结点 $\leq$ 左右孩子 $\rightarrow$ 堆顶最小。**最大堆：**每个结点 $\geq$ 左右孩子 $\rightarrow$ 堆顶最大。

```

1 import heapq
2
3 # ----- 最小堆（默认） -----
4 heap = []
5 heapq.heappush(heap, 5)
6 heapq.heappush(heap, 3)
7 heapq.heappush(heap, 7)
8 min_val = heapq.heappop(heap) # 3
9
10 # ----- 最大堆（取负） -----
11 max_heap = []
12 heapq.heappush(max_heap, -x) # 存负值
13 max_val = -heapq.heappop(max_heap) # 取负还原
14
15 # ----- 数组转堆 -----
16 arr = [4, 1, 7, 3, 9]
17 heapq.heapify(arr) # O(N) 原地建堆
18 # arr = [1, 3, 7, 4, 9]
19
20
21 # ----- Top-K（前 K 大 / 前 K 小） -----
22 def top_k_largest(nums, k):
23     """返回前 k 个最大元素"""
24     heap = []
25     for x in nums:
26         heapq.heappush(heap, x) # 最小堆，保持大小为 k
27         if len(heap) > k:
28             heapq.heappop(heap) # 弹出最小的，留下最大的 k 个
29     return heap # 注意：结果不是排序的
30
31 def top_k_smallest(nums, k):
32     """返回前 k 个最小元素"""
33     heap = [-x for x in nums[:k]]
34     heapq.heapify(heap)
35     for x in nums[k:]:
36         if -x > heap[0]: # x 比当前最大负值小
37             heapq.heapreplace(heap, -x)

```

```
38     return [-x for x in heap]
```

10 并查集（Union-Find，联合查找）

常用于判断图的连通分量、Kruskal 最小生成树等。

```
1 class UnionFind:
2     """并查集（路径压缩 + 按秩合并）"""
3     def __init__(self, n):
4         self.parent = list(range(n)) # 初始时每个节点指向自己
5         self.rank = [1] * n          # 树的高度（秩）
6         self.count = n               # 连通分量数
7
8     def find(self, x):
9         """查找 x 的根节点（路径压缩）"""
10        if self.parent[x] != x:
11            self.parent[x] = self.find(self.parent[x])
12        return self.parent[x]
13
14    def union(self, a, b):
15        """合并 a 和 b 所在的集合，返回是否合并成功"""
16        ra, rb = self.find(a), self.find(b)
17        if ra == rb:
18            return False # 已经在同一个集合
19        # 按秩合并：矮的树挂到高的树下
20        if self.rank[ra] < self.rank[rb]:
21            ra, rb = rb, ra
22        self.parent[rb] = ra
23        if self.rank[ra] == self.rank[rb]:
24            self.rank[ra] += 1
25        self.count -= 1 # 连通分量减 1
26        return True
27
28    def connected(self, a, b):
29        """判断 a 和 b 是否连通"""
30        return self.find(a) == self.find(b)
```

11 常用技巧与竞赛模板

11.1 求树的高度

```
1 def tree_height(root):
2     """求树的高度"""
```

```

3     if not root:
4         return 0
5     return max(tree_height(root.left), tree_height(root.right)) + 1

```

11.2 求树的直径（任意两节点最长路径）

```

1 def tree_diameter(root):
2     """求树中任意两个节点的最长路径（直径）"""
3     ans = 0
4
5     def dfs(node):
6         nonlocal ans
7         if not node:
8             return 0
9         l = dfs(node.left) # 左子树高度
10        r = dfs(node.right) # 右子树高度
11        ans = max(ans, l + r) # 经过当前节点的最长路径
12        return max(l, r) + 1 # 返回当前节点的高度
13
14    dfs(root)
15    return ans

```

11.3 判断是否为对称二叉树

```

1 def is_symmetric(root):
2     """判断二叉树是否关于根节点对称"""
3     def check(a, b):
4         if not a and not b:
5             return True
6         if not a or not b:
7             return False
8         return (a.val == b.val
9                 and check(a.left, b.right) # 左子树的左 vs 右子树的右
10                and check(a.right, b.left)) # 右子树的左 vs 左子树的右
11
12    return check(root.left, root.right) if root else True

```

11.4 判断是否为平衡二叉树

```

1 def is_balanced(root):
2     """判断二叉树是否为平衡二叉树"""
3     def dfs(node):
4         """返回高度，-1 表示不平衡"""

```

```

5     if not node:
6         return 0
7     l = dfs(node.left)
8     r = dfs(node.right)
9     if l == -1 or r == -1 or abs(l - r) > 1:
10        return -1
11    return max(l, r) + 1
12
13    return dfs(root) != -1

```

11.5 二叉树的镜像翻转

```

1 def invert_tree(root):
2     """翻转二叉树（镜像）"""
3     if not root:
4         return None
5     root.left, root.right = invert_tree(root.right), invert_tree(root.left)
6     return root

```

11.6 找二叉树的最近公共祖先（LCA, Lowest Common Ancestor）

```

1 def lowest_common_ancestor(root, p, q):
2     """
3     找两个节点的最近公共祖先
4     p 和 q 都在树中
5     """
6     if not root or root is p or root is q:
7         return root
8     left = lowest_common_ancestor(root.left, p, q)
9     right = lowest_common_ancestor(root.right, p, q)
10    if left and right: # p 和 q 分别在左右子树
11        return root
12    return left or right # 都在同一侧

```

11.7 二叉树的序列化与反序列化

```

1 def serialize(root):
2     """二叉树序列化（前序遍历，None 用 '#' 表示）"""
3     def dfs(node):
4         if not node:
5             res.append('#')
6             return

```



```

7         res.append(str(node.val))
8         dfs(node.left)
9         dfs(node.right)
10
11     res = []
12     dfs(root)
13     return ','.join(res)
14
15
16 def deserialize(data):
17     """反序列化还原二叉树"""
18     def dfs():
19         val = next(it)
20         if val == '#':
21             return None
22         node = TreeNode(int(val))
23         node.left = dfs()
24         node.right = dfs()
25         return node
26
27     it = iter(data.split(','))
28     return dfs()

```

12 笔试常见考点速查

- 三个结点能构成几种不同形态的二叉树？
5 种（Catalan 数，卡特兰数 $C_3 = 5$ ）
- 二叉树叶子数 n_0 和 n_2 的关系？
 $n_0 = n_2 + 1$
- 一棵完全二叉树有 n 个结点，叶子数？
 $n_0 = \lceil n/2 \rceil$ （或 $n_0 = \lfloor (n+1)/2 \rfloor$ ）
- 前序 + 中序能唯一确定一棵二叉树吗？
能。前序第一个为根，中序定位根后划分左右子树。
- 前序 + 后序能唯一确定一棵二叉树吗？
不能。当某结点只有左子树或只有右子树时无法区分。
- 完全二叉树用数组存，下标 i 的双亲、左右孩子？
双亲 $\lfloor i/2 \rfloor$ ，左孩子 $2i$ ，右孩子 $2i+1$ 。
- 哈夫曼树中度为 1 的结点数是多少？
0。哈夫曼树只有度为 0 和 2 的结点。

8. 最小堆的堆顶是什么？

最小值。最小堆中每个结点 $\leq$ 左右孩子。

9. 堆排序建堆的时间复杂度？

$O(N)$ 。用 Floyd（弗洛伊德）算法（从最后一个非叶子结点开始向下调整）。

10. 在有 n 个结点的 AVL 树中查找的最坏时间复杂度？

$O(\log n)$ 。因为 AVL 树高度为 $O(\log n)$ 。

13 Python 引用模块汇总（含常用代码模板）

考试中树相关的 Python 题目可能用到的所有模块及对应常用代码：

13.1 collections — 队列与字典

```
1 from collections import deque, defaultdict, Counter
2
3 # ----- deque: BFS（广度优先搜索）队列 -----
4 q = deque([root])
5 while q:
6     node = q.popleft() # O(1) 左端弹出
7     ...
8     if node.left: q.append(node.left)
9     if node.right: q.append(node.right)
10
11 # ----- deque: 双端操作 -----
12 dq = deque()
13 dq.append(x)    # 右端入
14 dq.appendleft(x) # 左端入
15 dq.pop()       # 右端出
16 dq.popleft()   # 左端出
17
18 # ----- defaultdict: 自动初始化 -----
19 adj = defaultdict(list)    # 邻接表: 默认空列表
20 depth = defaultdict(int)   # 默认 0
21
22 # ----- Counter: 计数 -----
23 freq = Counter(nums)
24 most = freq.most_common(1) # [(值, 次数)]
```

13.2 heapq — 堆

```
1 import heapq
2
```

```

3 heap = []
4 heapq.heappush(heap, x)    # O(log N) 入堆
5 x = heapq.heappop(heap)    # O(log N) 弹出最小
6 min_val = heap[0]          # O(1) 查看堆顶（不弹出）
7
8 # ----- 建堆 -----
9 heapq.heapify(arr)         # O(N) 原地建最小堆
10
11 # ----- 最大堆（取负） -----
12 heapq.heappush(heap, -x)
13 max_val = -heapq.heappop(heap)
14
15 # ----- 堆顶替换 -----
16 # heappushpop: 先 push 再 pop, 返回二者中较小的
17 # heapreplace: 先 pop 再 push, 返回原堆顶
18 x = heapq.heappushpop(heap, x) # 等价于 push + pop 但更高效
19 x = heapq.heapreplace(heap, x) # 等价于 pop + push 但更高效
20
21 # ----- Top-K（前 K 大 / 前 K 小） -----
22 heapq.nlargest(k, nums)    # 前 k 大
23 heapq.nsmallest(k, nums)   # 前 k 小

```

13.3 sys — 快速输入

```

1 import sys
2
3 # ----- 一次性读入所有整数 -----
4 data = list(map(int, sys.stdin.read().split()))
5 n = data[0]
6 arr = data[1:]
7
8 # ----- 逐行读入 -----
9 n = int(sys.stdin.readline())
10 arr = list(map(int, sys.stdin.readline().split()))
11
12 # ----- 多组数据 -----
13 for line in sys.stdin:
14     if not line.strip():
15         break
16     a, b = map(int, line.split())

```

13.4 itertools — 迭代工具

```

1 from itertools import accumulate, chain, permutations, combinations

```

```

2
3 # ----- 前缀和 -----
4 prefix = list(accumulate(arr)) # [a0, a0+a1, a0+a1+a2, ...]
5 prefix = list(accumulate(arr, initial=0)) # [0, a0, a0+a1, ...]
6
7 # ----- 排列组合 -----
8 list(permutations([1, 2, 3], 2)) # [(1,2), (1,3), (2,1), (2,3), (3,1), (3,2)]
9 list(combinations([1, 2, 3], 2)) # [(1,2), (1,3), (2,3)]
10
11 # ----- 展平嵌套 -----
12 list(chain.from_iterable([[1,2], [3]])) # [1, 2, 3]

```

13.5 functools — 函数式工具

```

1 from functools import lru_cache
2
3 # ----- 缓存递归结果（如斐波那契、树形 DP） -----
4 @lru_cache(maxsize=None)
5 def dfs(node_id, taken):
6     ...

```

13.6 math — 数学运算

```

1 import math
2
3 math.log2(n) # log2
4 math.log(n, 2) # 以 2 为底的对数
5 math.ceil(n / 2) # 向上取整
6 math.floor(n / 2) # 向下取整
7 math.inf # 无穷大（初始化最小值时用）

```

14 考试快速参考模板

```

1 # ===== 二叉树必备模板（复制即用） =====
2 from collections import deque
3 import heapq
4 import sys
5
6
7 class TreeNode:
8     def __init__(self, x):
9         self.val = x
10        self.left = None

```

```

11         self.right = None
12
13
14     def input_data():
15         """读取输入：第一个数是 n，后面跟两个长度为 n 的序列"""
16         data = list(map(int, sys.stdin.read().split()))
17         n = data[0]
18         seq1 = data[1:1 + n]
19         seq2 = data[1 + n:]
20         return n, seq1, seq2
21
22
23     def preorder(root):
24         return [] if not root else [root.val] + preorder(root.left) + preorder(root
                .right)
25
26     def inorder(root):
27         return [] if not root else inorder(root.left) + [root.val] + inorder(root.
                right)
28
29     def postorder(root):
30         return [] if not root else postorder(root.left) + postorder(root.right) + [
                root.val]
31
32     def levelorder(root):
33         if not root:
34             return []
35         res, q = [], deque([root])
36         while q:
37             node = q.popleft()
38             res.append(node.val)
39             if node.left: q.append(node.left)
40             if node.right: q.append(node.right)
41         return res
42
43
44     def build_pre_in(pre, ino):
45         """前序+中序 建树"""
46         if not pre or not ino:
47             return None
48         in_map = {v: i for i, v in enumerate(ino)}
49         def dfs(pl, pr, il, ir):
50             if pl > pr:
51                 return None
52             root = TreeNode(pre[pl])
53             idx = in_map[root.val]

```

```

54     left_size = idx - il
55     root.left = dfs(pl + 1, pl + left_size, il, idx - 1)
56     root.right = dfs(pl + left_size + 1, pr, idx + 1, ir)
57     return root
58 return dfs(0, len(pre) - 1, 0, len(ino) - 1)
59
60
61 def build_in_post(ino, post):
62     """中序+后序 建树"""
63     if not ino or not post:
64         return None
65     in_map = {v: i for i, v in enumerate(ino)}
66     def dfs(il, ir, pl, pr):
67         if pl > pr:
68             return None
69         root = TreeNode(post[pr])
70         idx = in_map[root.val]
71         left_size = idx - il
72         root.left = dfs(il, idx - 1, pl, pl + left_size - 1)
73         root.right = dfs(idx + 1, ir, pl + left_size, pr - 1)
74         return root
75     return dfs(0, len(ino) - 1, 0, len(post) - 1)
76
77
78 def post_to_level(seq):
79     """完全二叉树: 后序 -> 层序"""
80     n, idx = len(seq), 0
81     tree = [0] * (n + 1)
82     def dfs(i):
83         nonlocal idx
84         if i > n:
85             return
86         dfs(2 * i); dfs(2 * i + 1)
87         tree[i] = seq[idx]; idx += 1
88     dfs(1)
89     return tree[1:]
90
91
92 def height(root):
93     return 0 if not root else max(height(root.left), height(root.right)) + 1
94
95
96 def main():
97     n, seq1, seq2 = input_data()
98     root = build_pre_in(seq1, seq2) # 或 build_in_post(seq1, seq2)
99     print(' '.join(map(str, levelorder(root))))

```

```
100
101
102 if __name__ == '__main__':
103     main()
```